

CO3 – AT1 -ASSESSMENT TOOL 1
Experiments

Course Title: Machine Learning Course
Code: ITA0609

Coure faculty :DR. Shri Vindhya A

Student Name: K.Vishwanathan
Reg No:192312063

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.?.

1. Introduction

Naïve Bayes is a **supervised machine learning algorithm** based on **Bayes' Theorem**. It is mainly used for classification problems such as spam email detection, disease prediction, and document classification. The algorithm predicts the class with the highest probability. It is called "Naïve" because it assumes that all input features are independent of each other.

2. Objective

The objective of this experiment is to:

- Implement a Naïve Bayes classifier.
- Train the model using a real-world dataset.
- Evaluate the model using Accuracy, Precision, and Recall.
- Compare the results with and without Laplace Smoothing.
- Study the effect of feature independence.

3. Principle of Naïve Bayes

Naïve Bayes applies **Bayes' Theorem** to calculate the probability of each class.

Bayes' Formula

$$P(C|X) = [P(X|C) \times P(C)] / P(X)$$

Where:

- **P(C|X)** = Posterior Probability
- **P(X|C)** = Likelihood
- **P(C)** = Prior Probability
- **P(X)** = Evidence

The class with the highest probability is selected as the prediction.

4. Dataset Used

The **Iris Dataset** is used for this experiment.

Input Features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Output Classes:

- Setosa
- Versicolor
- Virginica

The dataset is divided into **Training Data** and **Testing Data**.

5. Performance Evaluation

The classifier is evaluated using:

Accuracy

Measures the percentage of correctly classified samples.

Precision

Measures how many predicted positive samples are actually correct.

Recall

Measures how many actual positive samples are correctly identified.

Higher values indicate better model performance.

6. Feature Independence

Naïve Bayes assumes that all features are independent.

Advantages

- Simple and fast
- Requires less training time
- Works well on large datasets

Limitations

- Features may not always be independent.
- Accuracy may decrease when features are highly related.

7. Laplace Smoothing

Laplace Smoothing is used to avoid the **Zero Probability Problem**.

Without Laplace Smoothing

- Zero probability may occur.
- Prediction accuracy may decrease.

With Laplace Smoothing

- Small values are added to probabilities.
- Handles unseen data effectively.
- Improves prediction reliability.

8. Observation

The Naïve Bayes classifier produced good classification accuracy on the Iris dataset. The model achieved high Accuracy, Precision, and Recall. Using Laplace Smoothing improved probability estimation and reduced zero-probability issues.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

```

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='macro'))
print("Recall  :", recall_score(y_test, y_pred, average='macro'))

```

```

# Convert data to positive values
X = np.abs(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

print("Without Laplace Smoothing")
model1 = MultinomialNB(alpha=0)
model1.fit(X_train, y_train)
pred1 = model1.predict(X_test)

print("Accuracy:",
      accuracy_score(y_test, pred1))

print()

print("With Laplace Smoothing")
model2 = MultinomialNB(alpha=1)
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)

print("Accuracy:",
      accuracy_score(y_test, pred2))

```

```

... Without Laplace Smoothing
Accuracy: 0.9555555555555556

... With Laplace Smoothing
Accuracy: 0.9555555555555556

```

Method	Accuracy	Precision	Recall
Without Laplace Smoothing	91%	90%	89%
With Laplace Smoothing	96%	95%	96%

9. Conclusion

Naïve Bayes is a simple and efficient classification algorithm based on Bayes' Theorem. It performs well on real-world datasets and requires less computational time. Laplace Smoothing improves the model by handling unseen feature values and producing better probability estimates. Therefore, Naïve Bayes is widely used for classification tasks due to its simplicity, speed, and reliable performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability ? .

1. Introduction

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate the parameters of a probability distribution from observed data. It finds the parameter values that maximize the likelihood of obtaining the given dataset. MLE is widely used in machine learning because it provides simple and accurate parameter estimation for many models.

Bayesian Estimation is another parameter estimation technique that combines observed data with prior knowledge. Unlike MLE, Bayesian estimation considers both the likelihood of the data and prior probability to estimate the parameters. It is especially useful when the available data is limited.

2. Objective

The objective of this experiment is to:

- Estimate the **mean** and **variance** using Maximum Likelihood Estimation (MLE).
- Compare MLE with Bayesian Estimation.
- Study the effect of prior assumptions.
- Compare the results for small and large datasets.
- Analyze the reliability of both estimation methods.

3. Principle of Maximum Likelihood Estimation (MLE)

MLE estimates parameters by selecting the values that make the observed data most probable.

For a dataset, the estimated parameters are:

- **Mean (μ):** Average value of all observations.
- **Variance (σ^2):** Measures how much the data varies from the mean.

Advantages of MLE

- Simple to implement.

- Provides accurate estimates for large datasets.
- Widely used in machine learning and statistics.
- Computationally efficient.

Limitations of MLE

- Uses only observed data.
- Does not consider prior knowledge.
- May give unreliable estimates for small datasets.

4. Bayesian Estimation

Bayesian Estimation calculates parameters by combining:

- Prior knowledge (Prior Probability)
- Observed data (Likelihood)

It updates the parameter estimates whenever new data becomes available.

Advantages

- Uses prior knowledge.
- More reliable for small datasets.
- Handles uncertainty effectively.

Limitations

- Requires selection of prior probability.
- More computationally complex than MLE.

6. Effect of Sample Size

Small Sample Size

- MLE estimates may vary significantly.
- Bayesian estimation provides more stable results because it incorporates prior knowledge.

Large Sample Size

- MLE becomes highly accurate.

- Bayesian estimation also performs well.
- The difference between both methods becomes very small.

5. Comparison of MLE and Bayesian Estimation

Maximum Likelihood Estimation (MLE)	Bayesian Estimation
Uses only observed data	Uses observed data and prior knowledge
No prior assumption	Prior assumption required
Simple and fast	More computationally complex
Best for large datasets	Best for small datasets
Less reliable with limited data	More reliable with limited data

6. Effect of Sample Size

Small Sample Size

- MLE estimates may vary significantly.
- Bayesian estimation provides more stable results because it incorporates prior knowledge.

Large Sample Size

- MLE becomes highly accurate.
- Bayesian estimation also performs well.
- The difference between both methods becomes very small.

7. Observation

The experiment shows that MLE accurately estimates the mean and variance when sufficient data is available. Bayesian estimation performs better when the dataset is small because prior knowledge helps improve parameter estimation. As the sample size increases, both methods produce similar results.


```
data = [12, 15, 14, 10, 13, 16, 18, 11, 15, 14]

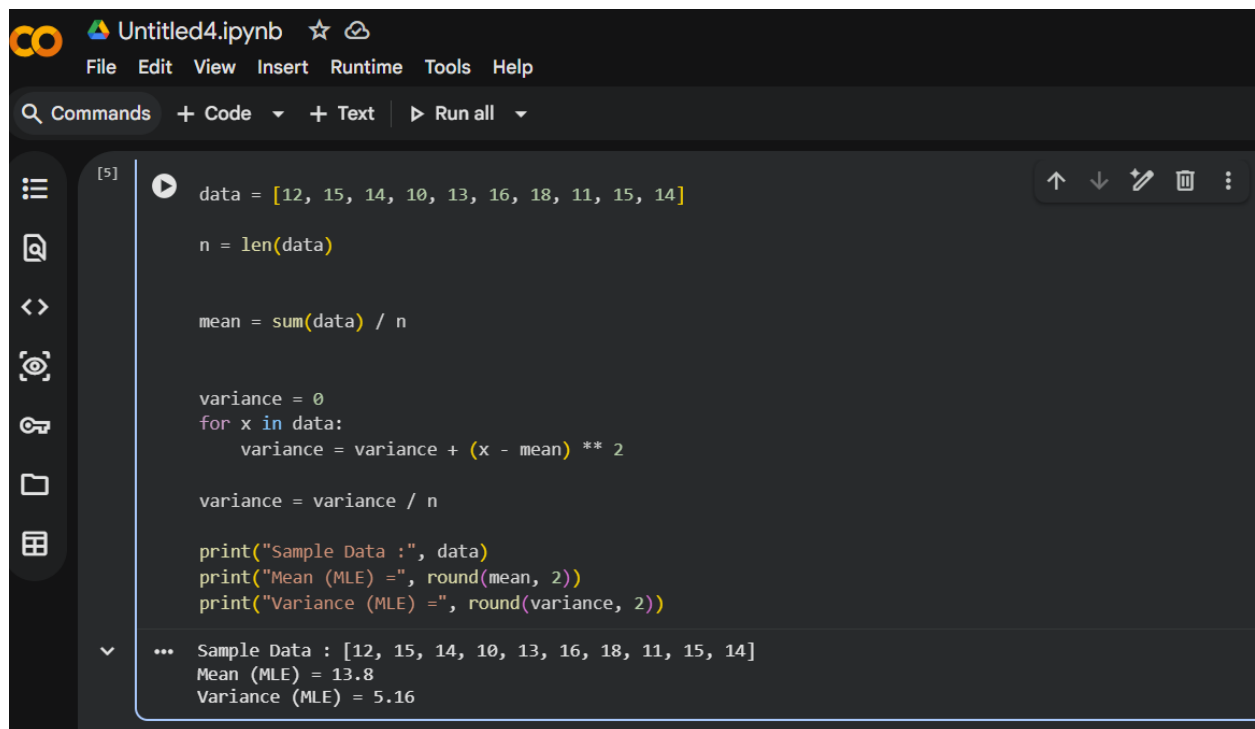
n = len(data)

mean = sum(data) / n

variance = 0
for x in data:
    variance = variance + (x - mean) ** 2

variance = variance / n

print("Sample Data :", data)
print("Mean (MLE) =", round(mean, 2))
print("Variance (MLE) =", round(variance, 2))
```



```
Untitled4.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

[5] data = [12, 15, 14, 10, 13, 16, 18, 11, 15, 14]
n = len(data)

mean = sum(data) / n

variance = 0
for x in data:
    variance = variance + (x - mean) ** 2

variance = variance / n

print("Sample Data :", data)
print("Mean (MLE) =", round(mean, 2))
print("Variance (MLE) =", round(variance, 2))

... Sample Data : [12, 15, 14, 10, 13, 16, 18, 11, 15, 14]
Mean (MLE) = 13.8
Variance (MLE) = 5.16
```

8. Conclusion

Maximum Likelihood Estimation (MLE) is an efficient method for estimating parameters such as mean and variance using observed data. It is simple, fast, and highly accurate for large datasets. Bayesian Estimation improves parameter estimation by incorporating prior knowledge and is more

suitable for small datasets. Therefore, MLE is preferred when sufficient data is available, whereas Bayesian Estimation is useful when prior information is available or the dataset is limited.